Learning and Findings from an FFT Ocean Simulation

Song Shi

August 2026

1 Introduction

Ocean surfaces contain waves at many spatial scales. Simulating the complete fluid dynamics is expensive, but a useful approximation can be obtained by representing the surface as a sum of periodic waves and evaluating that sum with an inverse fast Fourier transform (IFFT). In this report I describe my implementation of a GPU-based FFT ocean and record the ideas, mistakes, and practical findings that were most important while building it.

The implementation currently uses a seeded JONSWAP spectrum, deep-water dispersion, directional spreading, and GPU butterfly passes. It evolves 256×256 Fourier modes over a 128 m periodic domain and produces height, spectral slope, and horizontal-displacement fields. The main goals of this study are:

- to understand how an ocean wave spectrum becomes a two-dimensional Fourier representation of the surface;
- to implement and verify the time evolution and inverse FFT on the GPU;
- to understand which parameters control the shape and direction of the resulting waves; and
- to document the numerical and visual trade-offs of the method.

2 Spectral Ocean Model

2.1 Discrete ocean domain

Let the simulated ocean be a periodic square of side length L, sampled on an $N \times N$ grid. For grid indices $m, n \in \{0, \dots, N-1\}$, the centered wavevector is

$$\boldsymbol{k}_{mn} = \frac{2\pi}{L} \begin{bmatrix} m - N/2 \\ n - N/2 \end{bmatrix}. \tag{2.1}$$

The magnitude $k = \|\boldsymbol{k}\|$ determines the wavelength $\lambda = 2\pi/k$. Increasing N adds shorter waves, whereas increasing L increases the physical spacing between samples and changes the set of wavelengths represented by the grid.

2.2 Dispersion relation

Tessendorf begins with the incompressible Navier–Stokes equations and then makes several simplifying assumptions appropriate to ordinary open-ocean surface waves. The fluid is treated as incompressible and irrotational, so its velocity can be written as the gradient of a potential. Gravity is the only restoring force, atmospheric pressure at the surface is taken as constant, and the nonlinear quadratic term in Bernoulli's equation is discarded. The free surface is represented by a single-valued height field $h(\boldsymbol{x}, t)$ with small amplitude and slope. Finally, the water is treated as sufficiently deep that the solution decays below the surface without interacting with a bottom boundary.

Under these assumptions, the potential-flow equations can be reduced to the linear surface PDE used in the original notes,

$$\frac{\partial^4 h(\boldsymbol{x}, t)}{\partial t^4} = -g^2 \nabla_\perp^2 h(\boldsymbol{x}, t), \tag{2.2}$$

where $\nabla_\perp^2$ is the horizontal Laplacian. Because this PDE is linear, consider one plane-wave mode

$$h(\boldsymbol{x}, t) = h_0 \exp[i\,(\boldsymbol{k}\cdot\boldsymbol{x} - \omega t)]\,. \tag{2.3}$$

Substituting this mode into Equation 2.2 replaces the time derivatives by ω^4 and the horizontal Laplacian by $-k^2$, giving

$$h_0 \left(\omega^4 - g^2 k^2\right) = 0. \tag{2.4}$$

For a non-flat mode $h_0 \neq 0$, this requires $\omega^2 = gk$. The two signs of $\omega = \pm\sqrt{gk}$ describe opposite temporal phase directions; the implementation stores the positive angular-frequency magnitude and explicitly uses both $e^{i\omega t}$ and $e^{-i\omega t}$ during spectral evolution. Therefore, for deep water,

$$\omega(\boldsymbol{k}) = \sqrt{g\|\boldsymbol{k}\|}, \tag{2.5}$$

where g is gravitational acceleration. This relation determines how quickly each Fourier mode changes phase. The approximation fails when the depth d is not large compared with the wavelength; a finite-depth model instead introduces the bottom boundary and gives $\omega^2 = gk\tanh(kd)$. It also does not describe strongly nonlinear effects such as overturning or breaking waves.

2.3 JONSWAP spectrum and directionality

JONSWAP is an empirical model of a wind-driven sea whose waves have developed over a finite fetch. It is important to distinguish the spectrum from the random sampling used later. JONSWAP is not a probability distribution from which the program chooses individual frequencies, and the implementation does not sample a table of measured waves. Instead, it evaluates a continuous spectral density $S(\omega)$ that specifies how much surface-elevation variance is expected in a small interval of angular frequency. The measurements from the Joint North Sea Wave Project motivated the shape and parameters of this model. The program still draws its random values from a normal distribution. JONSWAP does not

replace that distribution; it determines the variance used to scale the normally distributed value in each Fourier bin.

The form evaluated by the implementation is

$$S(\omega) = \alpha g^2 \omega^{-5} \exp\left[-\frac{5}{4}\left(\frac{\omega_p}{\omega}\right)^4\right] \gamma^{r(\omega)}, \qquad r(\omega) = \exp\left[-\frac{(\omega-\omega_p)^2}{2\sigma^2\omega_p^2}\right], \tag{2.6}$$

where ω_p is the peak angular frequency, α controls the overall energy, and γ is the peak-enhancement factor. The implementation uses $\sigma = 0.07$ below the peak and $\sigma = 0.09$ above it, giving the peak its asymmetric measured shape. For wind speed U and fetch F, the code uses

$$\alpha = 0.076\left(\frac{U^2}{Fg}\right)^{0.22}, \qquad \omega_p = 22\left(\frac{g^2}{UF}\right)^{1/3}. \tag{2.7}$$

A longer fetch allows the wind to transfer energy to the water over a greater distance, while $\gamma > 1$ concentrates additional energy near the dominant frequency. Away from the peak, the model retains the broad ω^{-5} high-frequency decay of the underlying wind-sea spectrum.

This scalar spectrum states how much energy exists at each frequency, but not which horizontal direction a wave travels. A normalized spreading function $D(\theta, \omega)$ distributes that energy around the wind direction. The result must then be converted from frequency coordinates to the two-dimensional wavenumber grid used by the FFT. The conversion and subsequent random sampling are described together in the next subsection.

The active scene uses $U = 20\,\mathrm{m\,s^{-1}}$, $F = 100\,\mathrm{km}$, $\gamma = 6.0$, and a short-wave damping length of 0.35 m. Its directional model blends a broad wind-aligned distribution with a more focused frequency-dependent lobe using a blend factor of 0.85 and a swell-spread control of 0.2.

2.4 Random initial conditions

The JONSWAP value $S(\omega)$ is a variance density per angular-frequency interval, whereas each FFT coefficient represents one cell of a Cartesian wavenumber grid. It therefore cannot be used directly as the variance of a Fourier coefficient. For the deep-water relation $\omega = \sqrt{gk}$, the change from frequency to wavenumber introduces

$$\left|\frac{d\omega}{dk}\right| = \frac{g}{2\omega}. \tag{2.8}$$

The implementation first constructs the directional wavenumber density

$$P_h(\boldsymbol{k}) = \frac{2S(\omega)}{k}\left|\frac{d\omega}{dk}\right| D(\theta, \omega)\exp(-k^2\ell^2), \tag{2.9}$$

where D is normalized over direction, $1/k$ is the polar-to-Cartesian area factor, the factor two accounts for the paired propagation directions used by the real-valued surface construction, and $\exp(-k^2\ell^2)$ damps wavelengths near the grid limit. The zero mode is set to zero because the expression is singular there and a constant vertical offset is not an ocean wave.

This continuous density must then be integrated over one discrete FFT cell. On the square periodic domain,

$$\Delta k = \frac{2\pi}{L}, \qquad V_{\boldsymbol{k}} = P_h(\boldsymbol{k})(\Delta k)^2. \tag{2.10}$$

$V_{\boldsymbol{k}}$ is therefore the variance assigned to bin $\boldsymbol{k}$. The cell-area factor $(\Delta k)^2$ is essential: without it, changing L or N would change the total wave variance even if all physical spectrum parameters remained the same.

For every Fourier bin, the implementation independently samples two standard normal variables, $\xi_r, \xi_i \sim \mathcal{N}(0,1)$, and forms a complex Gaussian value $\xi(\boldsymbol{k}) = \xi_r + i\xi_i$. The initial amplitude is

$$\tilde{h}_0(\boldsymbol{k}) = \sqrt{\frac{V_{\boldsymbol{k}}}{2}}\,\xi(\boldsymbol{k}). \tag{2.11}$$

The normal distribution supplies random variation in amplitude and phase, while the spectrum-derived $V_{\boldsymbol{k}}$ sets the variance of that bin. Bins near the JONSWAP peak therefore tend to receive larger coefficients, but two runs can still produce different surfaces because their Gaussian samples differ. Using a fixed seed makes visual comparisons reproducible. Hermitian symmetry, $\tilde{h}(-\boldsymbol{k}) = \tilde{h}^*(\boldsymbol{k})$, is required so that the spatial height field is real-valued.

3 Time Evolution and Surface Fields

3.1 Spectrum evolution

The spectrum at time t is evaluated using

$$\tilde{h}(\boldsymbol{k}, t) = \tilde{h}_0(\boldsymbol{k})e^{i\omega t} + \tilde{h}_0^*(-\boldsymbol{k})e^{-i\omega t}. \tag{3.1}$$

The height field is obtained by applying the two-dimensional inverse Fourier transform,

$$h(\boldsymbol{x}, t) = \mathcal{F}^{-1}\left\{\tilde{h}(\boldsymbol{k}, t)\right\}. \tag{3.2}$$

3.2 Slope and horizontal displacement

Spatial derivatives can be calculated exactly for the represented modes by multiplication in Fourier space. The two slope spectra are

$$\widetilde{h_x} = ik_x\tilde{h}, \qquad \widetilde{h_z} = ik_z\tilde{h}. \tag{3.3}$$

Horizontal displacement is obtained in the same manner from a direction based on $\boldsymbol{k}/\|\boldsymbol{k}\|$. It can sharpen crests and flatten troughs, but too much displacement causes the parametrized surface to fold. Choppiness is disabled in the current baseline so that the height and slope implementation can first be evaluated independently.

4 GPU Implementation

4.1 Data flow

The initial spectrum is generated once and uploaded to the GPU. Each frame, a compute shader evaluates Equation 3.1 and constructs packed Fourier-domain fields. These fields

remain GPU-resident while a sequence of butterfly passes performs the two-dimensional IFFT. The final textures are then sampled by the ocean material to displace the mesh and reconstruct its shading normal.

The computation can be summarized as follows:

1. generate and upload the seeded initial spectrum;
2. evolve the spectrum for the current time;
3. construct height, slope, and displacement spectra;
4. execute horizontal butterfly stages;
5. execute vertical butterfly stages; and
6. unpack and sample the spatial fields during rendering.

4.2 Butterfly transform

For $N = 256$, a radix-2 transform requires $\log_2 N = 8$ stages in each dimension. A complete two-dimensional transform therefore uses 16 stages per packed field. Ping-pong render targets allow the output of one stage to become the input of the next without a CPU readback.

4.3 Rendering

The spatial fields displace a 512×512 plane. A physically based material receives direct illumination from one directional light, while an HDR environment supplies image-based lighting and reflections. Nearest filtering is used for FFT intermediates and linear filtering for the final spatial fields.

5 Findings and Learning

This section should contain the main contribution of the report. Organize it by finding rather than by the chronological order in which the code was written. For each finding, state the observation, explain its cause, and finish with its practical consequence.

5.1 Fourier conventions

Fourier conventions were among the most time-consuming conceptual aspects of this project. The original literature presents the Fourier representation compactly, as is appropriate for its intended audience, but leaves much of the supporting mathematical background implicit. Because neither Fourier analysis nor complex analysis was required in my undergraduate computer science curriculum, I could not take concepts such as complex exponentials, roots of unity, complex conjugation, Hermitian symmetry, or the relationship between the DFT and FFT for granted.

I therefore spent several days working backward from the equations and their implementation, with the aim of understanding why the operations are defined as they are rather than treating them as formulas to reproduce. Details that at first appeared arbitrary eventually followed from a small set of geometric and algebraic ideas: a complex exponential represents

rotation in the complex plane, the discrete Fourier basis functions represent different rates of rotation, and the FFT exploits symmetries among these rotations through an even–odd decomposition.

This subsection consequently gives more attention to several facts that may be elementary to readers with a background in applied mathematics or signal processing. Its purpose is not to provide a general introduction to Fourier analysis, but to record the minimum chain of reasoning that made the ocean simulation understandable from a computer science and graphics-programming perspective.

5.1.1 Why complex numbers appear

A complex number is useful here because it stores a two-dimensional quantity in a form whose algebra naturally describes rotation. The number

$$z = a + bi, \qquad i^2 = -1, \tag{5.1}$$

can be identified with the vector (a, b) in the complex plane. Its real part a is the horizontal coordinate, its imaginary part b is the vertical coordinate, and its magnitude is the Euclidean length $|z| = \sqrt{a^2 + b^2}$. If θ is the angle measured from the positive real axis, the same point can be written in polar form using Euler's identity:

$$z = |z|(\cos\theta + i\sin\theta) = |z|e^{i\theta}. \tag{5.2}$$

This form makes the role of the complex exponential especially clear. Multiplication by $e^{i\theta}$ rotates a point counterclockwise by θ without changing its length. In two-dimensional vector notation, the same operation is

$$(a + bi)e^{i\theta} \quad \longleftrightarrow \quad \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix}. \tag{5.3}$$

More generally, multiplying by $re^{i\theta}$ combines two transformations: the magnitude is scaled by r, and the direction is rotated by θ. Thus complex multiplication packages scaling and planar rotation into a single operation. This interpretation explains why complex numbers are natural in the Fourier transform: each Fourier component carries an amplitude and a phase, and its complex exponential advances that phase as a rotation in the complex plane.

5.1.2 From function spaces to the discrete Fourier basis

Before this project, I was already familiar with the idea that a periodic function, even one whose shape does not resemble a sinusoid, can be constructed from sinusoidal waves. A finite number of such waves gives an approximation, while additional frequencies reproduce progressively finer features. What I had not understood was why the Fourier formulation uses complex exponentials rather than working directly with sine and cosine waves in the real plane. Although the two descriptions are equivalent through Euler's identity, the complex form packages the sine and cosine components into a single rotating quantity.

More importantly, I had not connected this sum of waves to linear algebra, or recognized that functions themselves can be treated as vectors and sinusoidal functions as basis vectors in a function space. This perspective provides a useful bridge from ordinary finite-dimensional

linear algebra to Fourier analysis. Under the usual conditions for Fourier convergence, a periodic function can then be represented by an infinite series,

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ikx}. \tag{5.4}$$

This is where the description of Fourier analysis as an infinite-dimensional space became concrete for me. Each wave e^{ikx} supplies a different direction in a function space, and the full function may require infinitely many such directions. The coefficient c_k controls how much of the kth wave is present. The integer k describes how many rotations that wave completes as x traverses one period: positive and negative values rotate in opposite directions, while $k = 0$ gives the constant component.

After seeing the series this way, the connection to ordinary linear algebra provided a familiar framework for understanding it. In a finite-dimensional vector space, a vector is represented by coefficients relative to a chosen basis. For example,

$$\begin{bmatrix} 3 \\ 2 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 2 \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \tag{5.5}$$

The important fact is not the particular coordinates, but that the vector is a linear combination of basis vectors. Equation 5.4 has the same structure: the "vector" is now a function, the basis vectors are basis functions, and the coordinates are the Fourier coefficients c_k. Relating this unfamiliar function space to linear algebra, a subject with which I was already comfortable, made the underlying structure much easier to understand.

As with ordinary vectors, coefficients are easiest to separate when the basis is orthogonal. For complex-valued functions on $[0, 2\pi]$, an appropriate inner product is

$$\langle f, g \rangle = \frac{1}{2\pi} \int_0^{2\pi} f(x)\overline{g(x)}\, dx, \tag{5.6}$$

where the conjugate is the complex analogue of the transpose in a real dot product. If $\phi_k(x) = e^{ikx}$ and $\phi_m(x) = e^{imx}$, then

$$\langle \phi_k, \phi_m \rangle = \frac{1}{2\pi} \int_0^{2\pi} e^{i(k-m)x}\, dx = \begin{cases} 1, & k = m, \\ 0, & k \neq m. \end{cases} \tag{5.7}$$

When $k \neq m$, the exponential completes a nonzero integer number of rotations around the complex plane. Its contributions over the full period cancel, which is the geometric reason the integral is zero. The Fourier functions therefore play the same role as mutually perpendicular basis vectors in ordinary linear algebra.

The DFT appears when this continuous picture is sampled. A computer stores N values $f[0], \ldots, f[N-1]$, which form a vector in $\mathbb{C}^N$. Sampling one period at

$$x_n = \frac{2\pi n}{N}, \qquad n = 0, \ldots, N-1, \tag{5.8}$$

turns the continuous basis function into

$$e^{ikx_n} = e^{i2\pi kn/N}. \tag{5.9}$$

Here n selects a sample position and k selects a discrete frequency. It is convenient to name the rotation between adjacent samples

$$\omega_N = e^{i2\pi/N}. \tag{5.10}$$

The positive sign is the convention used here for the inverse-transform basis; the forward DFT commonly uses its conjugate, $e^{-i2\pi/N}$. Because $\omega_N^N = e^{i2\pi} = 1$, ω_N is an Nth root of unity. Its powers are the N equally spaced points on the unit circle, and Equation 5.9 becomes ω_N^{kn}. The kth sampled basis vector is therefore

$$\boldsymbol{b}_k = \begin{bmatrix} 1 & \omega_N^k & \omega_N^{2k} & \cdots & \omega_N^{(N-1)k} \end{bmatrix}^{\mathsf{T}}. \tag{5.11}$$

These vectors retain the orthogonality of the continuous Fourier functions. Their discrete inner product is

$$\langle \boldsymbol{b}_k, \boldsymbol{b}_m \rangle = \sum_{n=0}^{N-1} \omega_N^{(k-m)n} = \begin{cases} N, & k \equiv m \pmod N, \\ 0, & k \not\equiv m \pmod N. \end{cases} \tag{5.12}$$

For unequal frequency indices, the sum visits evenly spaced rotations whose vector sum is zero. The DFT basis is therefore not an unrelated construction for arrays: it is the finite-dimensional basis obtained by sampling the orthogonal complex exponentials of the continuous Fourier series.

Because these basis vectors use the positive exponential, they can be assembled as the columns of an unnormalized inverse, or synthesis, Fourier matrix,

$$B = \begin{bmatrix} \boldsymbol{b}_0 & \boldsymbol{b}_1 & \cdots & \boldsymbol{b}_{N-1} \end{bmatrix} = \begin{bmatrix} \omega_N^{nk} \end{bmatrix}_{n,k=0}^{N-1} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_N & \omega_N^2 & \cdots & \omega_N^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{N-1} & \omega_N^{2(N-1)} & \cdots & \omega_N^{(N-1)^2} \end{bmatrix}. \tag{5.13}$$

Column k is the sampled wave for frequency k, while row n records the value of every frequency at sample position n. Collect the Fourier coefficients into the vector $\boldsymbol{X}$, with $X_k = X[k]$. The standard inverse DFT is then the matrix-vector product

$$\boldsymbol{f} = \frac{1}{N} B \boldsymbol{X}, \qquad f[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \omega_N^{kn}. \tag{5.14}$$

Thus B is the unnormalized synthesis matrix, while $\frac{1}{N}B$ is the normalized IFFT operator. The IFFT reconstructs the sampled function by scaling each synthesis basis vector by its coefficient and adding the resulting vectors. Discrete orthogonality gives $B^{\mathsf{H}}B = NI$, where B^{H} is the conjugate transpose. This conjugate-transposed matrix uses the negative exponential and is the corresponding forward, or analysis, DFT matrix:

$$\boldsymbol{X} = B^{\mathsf{H}} \boldsymbol{f}, \qquad B^{-1} = \frac{1}{N} B^{\mathsf{H}}. \tag{5.15}$$

This matrix view also clarified the distinction between the DFT and FFT for me. The DFT and inverse DFT define the analysis and synthesis transformations, respectively, whereas

the FFT and IFFT compute those same matrix-vector results efficiently by exploiting the structure of the Fourier matrices rather than performing dense matrix multiplication.

5.1.3 FFT decomposition

The matrix formulation makes the computational problem explicit. Evaluating one output sample of an N-point DFT or inverse DFT requires a sum over all N input coefficients. Repeating this for all N outputs therefore requires $O(N^2)$ complex operations. For the two-dimensional ocean grid, applying this directly along both dimensions would be too expensive to repeat every frame.

The FFT reduces this cost by exploiting a symmetry in polynomial evaluation. Using the positive-exponent synthesis convention, define the polynomial

$$P(z) = \sum_{k=0}^{N-1} X[k] z^k. \tag{5.16}$$

The unnormalized inverse DFT evaluates this polynomial at the roots of unity, $\widetilde{f}[n] = P(\omega_N^n)$; the standard normalized result is $f[n] = \widetilde{f}[n]/N$. The polynomial can be separated into its even- and odd-indexed coefficients,

$$P(z) = P_e(z^2) + zP_o(z^2), \tag{5.17}$$

where

$$P_e(u) = \sum_{r=0}^{N/2-1} X[2r] u^r, \qquad P_o(u) = \sum_{r=0}^{N/2-1} X[2r+1] u^r. \tag{5.18}$$

The factor z on the odd branch is not an additional rule introduced by the FFT. It follows directly from factoring an odd power: $z^{2r+1} = z(z^2)^r$. At the nth root of unity, this factor becomes the twiddle factor ω_N^n, so

$$P(\omega_N^n) = P_e(\omega_N^{2n}) + \omega_N^n P_o(\omega_N^{2n}). \tag{5.19}$$

The squared root satisfies

$$\omega_N^2 = e^{i4\pi/N} = e^{i2\pi/(N/2)} = \omega_{N/2}. \tag{5.20}$$

Consequently, both P_e and P_o are evaluated at the roots of unity for an $N/2$-point transform. The original problem has become two transforms of half the size. Moreover, $\omega_N^{n+N/2} = -\omega_N^n$, while its square is unchanged. One pair of half-size results $E_n = P_e(\omega_{N/2}^n)$ and $O_n = P_o(\omega_{N/2}^n)$ therefore produces two full-size outputs:

$$\begin{aligned} P(\omega_N^n) &= E_n + \omega_N^n O_n, \\ P(\omega_N^{n+N/2}) &= E_n - \omega_N^n O_n. \end{aligned} \tag{5.21}$$

This paired sum and difference is the butterfly operation. The even result is reused for both outputs, while the odd result is first rotated by the twiddle factor and then either added or subtracted. Applying the same even–odd split recursively gives

$$T(N) = 2T(N/2) + O(N) = O(N \log_2 N), \tag{5.22}$$

for a power-of-two transform. The FFT is therefore the same Fourier matrix-vector multiplication as before, reorganized to reuse intermediate results.

The GPU implementation executes this recursion iteratively. The input spectrum is first read in bit-reversed order, placing coefficients in the arrangement expected by an iterative radix-2 decimation-in-time transform. Stage s then combines groups of span 2^{s+1}: each shader invocation reads an even value and an odd value, multiplies the odd value by the stage twiddle factor, and writes either their sum or their difference. Ping-pong render targets preserve the output of one stage as the input to the next without a CPU readback.

The slope field provides a concrete example of why the same transform can be reused. Writing the two-dimensional height synthesis schematically as

$$h(\boldsymbol{r}) = \frac{1}{N^2} \sum_{\boldsymbol{k}} \widetilde{h}(\boldsymbol{k}) e^{i\boldsymbol{k}\cdot\boldsymbol{r}}, \qquad \boldsymbol{r} = (x, z), \tag{5.23}$$

differentiating with respect to x gives

$$\frac{\partial h}{\partial x} = \frac{1}{N^2} \sum_{\boldsymbol{k}} i k_x \widetilde{h}(\boldsymbol{k}) e^{i\boldsymbol{k}\cdot\boldsymbol{r}} = \mathrm{IFFT}_2\Big\{ i k_x \widetilde{h} \Big\}. \tag{5.24}$$

The exponential basis function is unchanged; differentiation only multiplies its coefficient by ik_x. Similarly, $\partial h/\partial z = \mathrm{IFFT}_2\{ik_z\widetilde{h}\}$. The height and both slopes therefore require different spectral inputs but exactly the same IFFT operation. Once the slopes are spatial fields, the normal of the height graph is proportional to $(-\partial h/\partial x, 1, -\partial h/\partial z)$.

Because the ocean transform is two-dimensional, the same $\log_2 N$ stages are first applied horizontally and then vertically. Importantly, the FFT algorithm does not depend on what a spectral field represents. It performs the same linear transformation on the height spectrum $\widetilde{h}$, the slope spectra $ik_x\widetilde{h}$ and $ik_z\widetilde{h}$, and the horizontal-displacement spectra. The implementation can therefore reuse one butterfly shader for every ocean field rather than requiring a separate transform for height, slopes, or displacement.

Where possible, two complex spectra are packed into the RG and BA channel pairs of one texture, allowing each butterfly invocation to transform both fields in parallel. After the inverse transforms, the vertex shader samples the spatial height and slope fields. It displaces the mesh with the height field and constructs the surface normal from the two slope components; a separate normal field does not need its own inverse transform. The butterfly stages compute an unnormalized synthesis transform. Under the standard inverse-DFT convention, normalization can be applied after the final stage as $1/N$ for a one-dimensional transform or $1/N^2$ after both dimensions.

6 Limitations and Future Work

The current model assumes linear deep-water waves on a periodic domain. It does not model breaking waves, shoreline interaction, currents, variable depth, or two-way coupling with objects. The next planned field is the displacement Jacobian, which can identify folding and drive foam generation. Other possible extensions include cascaded FFT domains, shallow-water dispersion, persistent foam, spray, and GPU timing instrumentation.

7 Conclusion

This project implemented a GPU-based spectral ocean simulation using a JONSWAP spectrum, deep-water dispersion, and a two-dimensional inverse FFT. The initial Fourier coefficients are generated from complex Gaussian samples whose variances are determined by the ocean spectrum. Their phases are then evolved over time, and GPU butterfly passes reconstruct the height and slope fields. The result is a periodic ocean surface whose statistical form is controlled in frequency space and reconstructed efficiently in the spatial domain.

The most important outcome was not only the rendered surface, but a clearer understanding of the mathematical structure behind it. Complex exponentials represent amplitude and phase in a single quantity; Fourier basis functions extend the familiar idea of orthogonal basis vectors from finite-dimensional linear algebra to function spaces; and the DFT returns this idea to a finite setting by sampling those basis functions. From this perspective, the IFFT is a synthesis operation that combines frequency-domain coefficients into spatial samples, while the FFT is an efficient decomposition of the same transform.

Several implementation details that initially appeared unrelated followed from this structure. Hermitian symmetry is required to obtain a real-valued surface; spectral differentiation allows height, slope, and displacement fields to reuse the same IFFT; and JONSWAP does not replace Gaussian sampling, but determines the variance assigned to each Fourier bin. The project also clarified the difference between spectral and geometric resolution: increasing mesh density cannot create wavelengths that are absent from the Fourier grid.

The current implementation remains a linear, periodic deep-water model and does not represent changing depth, shoreline interaction, wave breaking, or two-way coupling with objects. Possible extensions include using the displacement Jacobian to identify folding and drive foam, or combining multiple spectral cascades to cover a wider range of wavelengths. Before increasing the model's complexity, the existing transform could also be validated more systematically with single-frequency inputs and quantitative CPU–GPU comparisons.

References

[1] J. Tessendorf, "Simulating Ocean Water," in *SIGGRAPH Course Notes*, 2001.

[2] K. Hasselmann et al., "Measurements of wind-wave growth and swell decay during the Joint North Sea Wave Project (JONSWAP)," *Ergänzungsheft zur Deutschen Hydrographischen Zeitschrift*, Series A, no. 12, 1973.